```
# Install all required packages
!pip install -q xgboost scikit-learn pandas numpy matplotlib seaborn
!pip install -q textblob vaderSentiment
!pip install -q transformers accelerate bitsandbytes sentencepiece
!pip install -q torch torchvision torchaudio

# Download NLTK data needed by TextBlob
import nltk
nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('punkt_tab', quiet=True)

# Create project folders
import os
for folder in ['data', 'models', 'outputs']:
    os.makedirs(folder, exist_ok=True)

print("✓ All packages installed and folders created")
```

```
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 126.0/126.0 kB 6.1 MB/s eta 0:00:00
   ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 60.7/60.7 MB 14.5 MB/s eta 0:00:00
✓ All packages installed and folders created
```

```
import torch
print("GPU available:", torch.cuda.is_available())
print("GPU name:     ", torch.cuda.get_device_name(0) if torch.cuda.is_available() else "None — go to Runtime → Change runt
print("VRAM:         ", round(torch.cuda.get_device_properties(0).total_memory/1e9, 1), "GB" if torch.cuda.is_available() e
```

```
GPU available: True
GPU name:      Tesla T4
VRAM:          15.6 GB
```

```
# CELL 3 (COMPLETE FIX) — Upload + verify + fix name all in one

import os, shutil, zipfile
from google.colab import files

# 1. Create folders
os.makedirs("data", exist_ok=True)
os.makedirs("models", exist_ok=True)
os.makedirs("outputs", exist_ok=True)

# 2. Upload file
print("Select your file — can be the CSV or the ZIP from Kaggle\n")
uploaded = files.upload()

# 3. Handle whatever was uploaded
for fname in uploaded.keys():
    print(f"\nYou uploaded: {fname}")

    if fname.endswith(".zip"):
        # Extract zip
        with zipfile.ZipFile(fname, "r") as z:
            z.extractall("data/")
            print(f"Extracted: {z.namelist()}")
        os.remove(fname)

    elif fname.endswith(".csv"):
        # Move CSV to data/
        shutil.move(fname, f"data/{fname}")
        print(f"Moved to data/{fname}")

# 4. Find the CSV and rename it to expected name
csv_found = False
for f in os.listdir("data/"):
    if f.endswith(".csv"):
        if f != "fake_reviews_dataset.csv":
            os.rename(f"data/{f}", "data/fake_reviews_dataset.csv")
            print(f"Renamed '{f}' → 'fake_reviews_dataset.csv'")
        csv_found = True
        break

# 5. Final check
if csv_found:
    size_mb = os.path.getsize("data/fake_reviews_dataset.csv") / 1e6
    print(f"\n✓ data/fake_reviews_dataset.csv is ready ({size_mb:.1f} MB)")
    print("  You can now run Cell 4 (feature engineering)")
else:
    print("\n✗ No CSV found. Please re-upload the file.")
```

```
Select your file — can be the CSV or the ZIP from Kaggle

Choose Files fake reviews dataset.csv
fake reviews dataset.csv(text/csv) - 15324616 bytes, last modified: 3/21/2026 - 100% done
Saving fake reviews dataset.csv to fake reviews dataset.csv

You uploaded: fake reviews dataset.csv
Moved to data/fake reviews dataset.csv
Renamed 'fake reviews dataset.csv' → 'fake_reviews_dataset.csv'

✓ data/fake_reviews_dataset.csv is ready (15.3 MB)
  You can now run Cell 4 (feature engineering)
```

```
# Re-install packages after session restart
!pip install -q xgboost scikit-learn textblob vaderSentiment matplotlib seaborn transformers accelerate bitsandbytes senter

import nltk
nltk.download('punkt', quiet=True)
nltk.download('averaged_perceptron_tagger', quiet=True)
nltk.download('punkt_tab', quiet=True)

print("✓ Packages reinstalled")
```

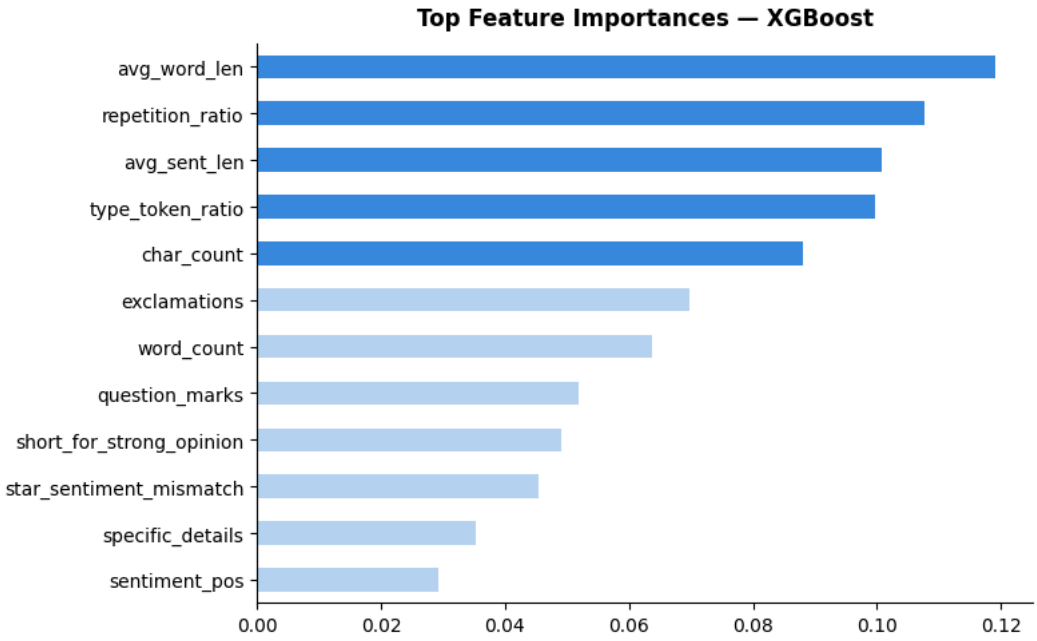
```
✓ Packages reinstalled
```

```
import pandas as pd, numpy as np, re, pickle
from textblob import TextBlob
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
from sklearn.model_selection import train_test_split

vader = SentimentIntensityAnalyzer()
VAGUE = re.compile(r'\b(best|worst|amazing|perfect|fantastic|incredible|horrible|terrible|awful|outstanding|excellent|superb
SPECIFIC = re.compile(r'\b(\d+|month|week|day|hour|year|cm|mm|inch|kg|lb|model|version|serial|update|warranty|refund|return|

def extract_text_features(text):
    text = str(text); words = text.split()
    sents = [s.strip() for s in re.split(r'[.!?]+', text) if s.strip()]
    blob = TextBlob(text); vs = vader.polarity_scores(text)
    n_words = len(words); n_sents = max(len(sents),1)
    caps = sum(1 for w in words if len(w)>2 and w.isupper())
    uniq = len({w.lower() for w in words})
    wf = {}
    for w in words: wf[w.lower()] = wf.get(w.lower(),0)+1
    repeated = sum(1 for c in wf.values() if c>2)
    return {"word_count":n_words,"char_count":len(text),"avg_word_len":round(np.mean([len(w) for w in words]) if words else

def extract_behavioral_features(row):
    rating=float(row.get("rating",3)); text=str(row.get("text_",""))
    vs=vader.polarity_scores(text); sent=vs["compound"]; wc=len(text.split())
    return {"rating":rating,"star_sentiment_mismatch":int((rating>=4 and sent<-0.2)or(rating<=2 and sent>0.2)),"is_extreme_r

FEATURE_COLS = ["word_count","char_count","avg_word_len","avg_sent_len","exclamations","question_marks","caps_ratio","vague_

print("Loading dataset...")
df = pd.read_csv("data/fake_reviews_dataset.csv")
df.columns = [c.strip().lower().replace(" ","_") for c in df.columns]
df.dropna(subset=["text_","label"], inplace=True)
df.reset_index(drop=True, inplace=True)
df["label_binary"] = df["label"].str.strip().map({"CG":1,"OR":0})
df.dropna(subset=["label_binary"], inplace=True)
df["label_binary"] = df["label_binary"].astype(int)
print(f"Loaded {len(df):,} reviews | Labels:\n{df['label_binary'].value_counts().to_string()}")

print("\nExtracting features (takes ~2-3 min)...")
tf = df["text_"].apply(extract_text_features).apply(pd.Series)
bf = df.apply(extract_behavioral_features, axis=1).apply(pd.Series)
fdf = pd.concat([tf,bf],axis=1)
fdf["label"]=df["label_binary"].values; fdf["text_"]=df["text_"].values; fdf["rating"]=df["rating"].values

X,y = fdf.drop(columns=["label"]), fdf["label"]
Xtmp,Xte,ytmp,yte = train_test_split(X,y,test_size=0.15,random_state=42,stratify=y)
Xtr,Xvl,ytr,yvl = train_test_split(Xtmp,ytmp,test_size=0.176,random_state=42,stratify=ytmp)

for name,obj in [("X_train",Xtr),("X_val",Xvl),("X_test",Xte),("y_train",ytr),("y_val",yvl),("y_test",yte)]:
    obj.to_pickle(f"data/{name}.pkl")
with open("data/feature_cols.pkl","wb") as f: pickle.dump(FEATURE_COLS,f)

print(f"\n✓ Feature engineering done!")
print(f"  Train: {len(Xtr):,} | Val: {len(Xvl):,} | Test: {len(Xte):,}")
print(f"  Features extracted: {len(FEATURE_COLS)}")
```

```
Loading dataset...
Loaded 40,432 reviews | Labels:
label_binary
1    20216
0    20216

Extracting features (takes ~2-3 min)...

✓ Feature engineering done!
  Train: 28,318 | Val: 6,049 | Test: 6,065
  Features extracted: 19
```

```
import pickle, numpy as np, pandas as pd
from sklearn.linear_model import LogisticRegression
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import classification_report, roc_auc_score, accuracy_score, confusion_matrix
from xgboost import XGBClassifier
import matplotlib.pyplot as plt, seaborn as sns

with open("data/feature_cols.pkl","rb") as f: FEATURE_COLS=pickle.load(f)
Xtr=pd.read_pickle("data/X_train.pkl"); Xvl=pd.read_pickle("data/X_val.pkl"); Xte=pd.read_pickle("data/X_test.pkl")
ytr=pd.read_pickle("data/y_train.pkl"); yvl=pd.read_pickle("data/y_val.pkl"); yte=pd.read_pickle("data/y_test.pkl")

# --- Model A: XGBoost ---
print("Training XGBoost...")
scaler = StandardScaler()
Xtr_s=scaler.fit_transform(Xtr[FEATURE_COLS]); Xvl_s=scaler.transform(Xvl[FEATURE_COLS])
xgb = XGBClassifier(n_estimators=300,max_depth=6,learning_rate=0.05,subsample=0.8,colsample_bytree=0.8,eval_metric="logloss"
xgb.fit(Xtr_s,ytr,eval_set=[(Xvl_s,yvl)],verbose=50)

# --- Model B: TF-IDF + LR ---
print("\nTraining TF-IDF + Logistic Regression...")
tfidf = Pipeline([("tfidf",TfidfVectorizer(ngram_range=(1,2),max_features=8000,sublinear_tf=True,min_df=3)),("clf",LogisticR
tfidf.fit(Xtr["text_"].fillna(""),ytr)

# --- Ensemble on test set ---
Xte_s=scaler.transform(Xte[FEATURE_COLS])
xgb_p=xgb.predict_proba(Xte_s)[:,1]
tfidf_p=tfidf.predict_proba(Xte["text_"].fillna(""))[:,1]
ens_p=0.6*xgb_p+0.4*tfidf_p

print("\n========== TEST SET RESULTS ==========")
for name,p in [("XGBoost",xgb_p),("TF-IDF+LR",tfidf_p),("Ensemble",ens_p)]:
    print(f"\n--- {name} ---")
    print(classification_report(yte,(p>=0.5).astype(int),target_names=["Genuine","Fake"],digits=4))
    print(f"AUC: {roc_auc_score(yte,p):.4f}")

# Save models
with open("models/xgb_model.pkl","wb") as f: pickle.dump(xgb,f)
with open("models/xgb_scaler.pkl","wb") as f: pickle.dump(scaler,f)
with open("models/tfidf_pipeline.pkl","wb") as f: pickle.dump(tfidf,f)
np.save("data/test_ensemble_probs.npy",ens_p)

# Feature importance chart
imp=pd.Series(xgb.feature_importances_,index=FEATURE_COLS).sort_values(ascending=True).tail(12)
fig,ax=plt.subplots(figsize=(8,5))
colors=["#378ADD" if i>=len(imp)-5 else "#B5D4F4" for i in range(len(imp))]
imp.plot(kind="barh",ax=ax,color=colors,edgecolor="none")
ax.set_title("Top Feature Importances — XGBoost",fontweight="bold",pad=10)
ax.spines[["top","right"]].set_visible(False)
plt.tight_layout(); plt.savefig("outputs/feature_importance.png",dpi=150); plt.show()
print("\n✓ Models saved to models/  |  Chart saved to outputs/")
```

```
Training XGBoost...
[0] validation_0-logloss:0.67412
[50]    validation_0-logloss:0.38448
[100]   validation_0-logloss:0.34729
[150]   validation_0-logloss:0.33606
[200]   validation_0-logloss:0.33152
[250]   validation_0-logloss:0.32901
[299]   validation_0-logloss:0.32738

Training TF-IDF + Logistic Regression...

========== TEST SET RESULTS ==========

--- XGBoost ---
              precision    recall  f1-score   support

     Genuine     0.8534    0.8444    0.8489      3033
        Fake     0.8460    0.8549    0.8504      3032

    accuracy                         0.8496      6065
   macro avg     0.8497    0.8496    0.8496      6065
weighted avg     0.8497    0.8496    0.8496      6065

AUC: 0.9347

--- TF-IDF+LR ---
              precision    recall  f1-score   support

     Genuine     0.9289    0.9430    0.9359      3033
        Fake     0.9421    0.9278    0.9349      3032

    accuracy                         0.9354      6065
   macro avg     0.9355    0.9354    0.9354      6065
weighted avg     0.9355    0.9354    0.9354      6065

AUC: 0.9833

--- Ensemble ---
              precision    recall  f1-score   support

     Genuine     0.9187    0.9097    0.9142      3033
        Fake     0.9105    0.9195    0.9150      3032

    accuracy                         0.9146      6065
   macro avg     0.9146    0.9146    0.9146      6065
weighted avg     0.9146    0.9146    0.9146      6065

AUC: 0.9793
```




```
✓ Models saved to models/  |  Chart saved to outputs/
```

```
# CELL 7: Live Sample Testing
import pickle, re, numpy as np, pandas as pd
from textblob import TextBlob
from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer

print("Loading saved models...")
with open("data/feature_cols.pkl","rb") as f: FEATURE_COLS = pickle.load(f)
with open("models/xgb_model.pkl","rb") as f: xgb_m = pickle.load(f)
with open("models/xgb_scaler.pkl","rb") as f: scl = pickle.load(f)
```

```
with open("models/tfidf_pipeline.pkl","rb") as f: tfidf_m = pickle.load(f)

vader = SentimentIntensityAnalyzer()
VAGUE = re.compile(r'\b(best|worst|amazing|perfect|fantastic|incredible|horrible|terrible|awful|outstanding|excellent|superb
SPECIFIC = re.compile(r'\b(\d+|month|week|day|hour|year|cm|mm|inch|kg|lb|model|version|serial|update|warranty|refund|return|

def extract_single_feature_set(text, rating=5.0):
    words = text.split()
    sents = [s.strip() for s in re.split(r'[.!?]+', text) if s.strip()]
    blob = TextBlob(text)
    vs = vader.polarity_scores(text)
    caps = sum(1 for w in words if len(w)>2 and w.isupper())
    uniq = len({w.lower() for w in words})

    feats = {
        "word_count": len(words), "char_count": len(text),
        "avg_word_len": round(np.mean([len(w) for w in words]) if words else 0, 3),
        "avg_sent_len": round(len(words)/max(len(sents), 1), 3),
        "exclamations": text.count("!"), "question_marks": text.count("?"),
        "caps_ratio": round(caps/max(len(words), 1), 4),
        "vague_superlatives": len(VAGUE.findall(text)), "specific_details": len(SPECIFIC.findall(text)),
        "type_token_ratio": round(uniq/max(len(words), 1), 4), "repetition_ratio": 0.0,
        "sentiment_compound": round(vs["compound"], 4), "sentiment_pos": round(vs["pos"], 4),
        "sentiment_neg": round(vs["neg"], 4), "subjectivity": round(blob.sentiment.subjectivity, 4),
        "polarity": round(blob.sentiment.polarity, 4), "rating": float(rating),
        "star_sentiment_mismatch": int((rating>=4 and vs["compound"]<-0.2) or (rating<=2 and vs["compound"]>0.2)),
        "is_extreme_rating": int(rating in [1.0, 5.0]),
        "short_for_strong_opinion": int(len(words)<20 and rating in [1.0, 5.0])
    }
    return pd.DataFrame([feats])[FEATURE_COLS]

def test_review(text, rating=5.0):
    # 1. Feature Extraction & XGBoost Score
    feats_df = extract_single_feature_set(text, rating)
    xgb_score = xgb_m.predict_proba(scl.transform(feats_df))[0, 1]

    # 2. TF-IDF Text Score
    tfidf_score = tfidf_m.predict_proba([text])[0, 1]

    # 3. Blended Ensemble Score (60% Behavior/Lexical, 40% Raw Text)
    final_score = (0.6 * xgb_score) + (0.4 * tfidf_score)
    prediction = "🚨 FAKE / DECEPTIVE" if final_score >= 0.50 else "✅ GENUINE"

    print(f"Review: \"{text}\"")
    print(f"Rating: {rating} Stars")
    print(f"Score Breakdown -> XGBoost: {xgb_score:.3f} | TF-IDF: {tfidf_score:.3f}")
    print(f"Final Score: {final_score:.3f} ---> {prediction}\n")
    print("-" * 70)

print("\n" + "="*50)
print("LIVE FAKE REVIEW DETECTOR")
print("="*50 + "\n")

# --- TEST YOUR OWN REVIEWS HERE ---

# Example 1: Obvious Fake (High caps, superlatives, no details)
test_review("BEST PRODUCT EVER!!! I cannot believe how AMAZING this is!!! PERFECT IN EVERY WAY!!! BUY NOW!!!", 5.0)

# Example 2: Genuine (Nuanced, specific details, mixed sentiment)
test_review("this is  a great product. i have used this many times.")
# Example 3: Suspicious/Bot-like (Extremely short for a 5-star review)
test_review("Good product. Recommended.", 5.0)
```

```
Loading saved models...

==================================================
LIVE FAKE REVIEW DETECTOR
==================================================

Review: "BEST PRODUCT EVER!!! I cannot believe how AMAZING this is!!! PERFECT IN EVERY WAY!!! BUY NOW!!!"
Rating: 5.0 Stars
Score Breakdown -> XGBoost: 0.293 | TF-IDF: 0.036
Final Score: 0.190 ---> ✅ GENUINE

----------------------------------------------------------------------
Review: "this is  a great product. i have used this many times."
Rating: 5.0 Stars
Score Breakdown -> XGBoost: 0.189 | TF-IDF: 0.702
Final Score: 0.394 ---> ✅ GENUINE

----------------------------------------------------------------------
Review: "Good product. Recommended."
Rating: 5.0 Stars
Score Breakdown -> XGBoost: 0.402 | TF-IDF: 0.203
```